

API Automation Framework

Architecture, Use Cases & Execution
Flow

Project Description

- Java-based API automation framework
- Cucumber JVM for behavior definition
- TestNG for execution lifecycle
- Custom DSL for API testing
- WireMock for service virtualization
- Allure for reporting, Maven for build

High-Level Project Structure

- `src/main/java` – framework and DSL
- `src/test/java` – runners, step definitions, hooks
- `src/test/resources` – feature files, mocks, test data

Framework Layer

- Configuration handling
- Custom DSL for API calls
- ScenarioContext for shared state
- WireMock lifecycle helper

Execution Layer

- TestNG runner (CliIT)
- Cucumber step definitions
- Hooks for setup and teardown
- Allure listeners

Use Case – Favorite Vendor API

- Retrieve favorite vendor for a player
- Validate response structure
- Assert business rules
- Data-driven via Scenario Outline

Use Case – Payments & Refunds

- Payment retrieval scenarios
- Refund workflow validation
- Negative and edge cases
- Downstream services mocked

Execution Flow – End to End

- Maven / CI triggers execution
- TestNG initializes runner
- Cucumber executes scenarios
- DSL performs API calls
- WireMock mocks dependencies
- Allure reports results

Q&A – Common Questions

- Why Cucumber + TestNG?
- Why a custom DSL?
- How are dependencies isolated?
- Is this framework scalable?
- How does it fit into CI/CD?